WispyScheme: Metacircularity as a Theorem

Functional Pearl: one law, six machines, and an eight-element algebra

STEFANO PALMIERI, Independent researcher,

Every Lisp rests on one law: evaluating a quotation recovers the program, `(eval (quote p))` behaves as `p`. The Scheme reports stipulate it, every implementation implements it, and every metacircular evaluator exhibits it — but the law is always an axiom of the evaluator that satisfies it. We present a small Scheme dialect, WispyScheme, in which the metacircular law is a *theorem*. Its quote and eval are underwritten by an eight-element algebra — itself derived, by enumerating the models of a law set and taking the lexicographically minimal one — whose certified retraction law becomes the base case of the metacircularity proof. We then rebuild the evaluator five times: adding environments, closures and the fuel that β-reduction forces, first-class continuations, a mutable store, and pairs with dispatch, ending in a call-by-value sequent-calculus machine on which quotations are first-class constructible data. Each extension is proved conservative over the last, and the metacircular law survives every restructuring unchanged, uniformly in fuel: programs and their quotations converge together, diverge together, and agree on every value. The development is mechanized in Lean 4 (98 theorems for the ladder, zero sorry, no axiom beyond propositional extensionality) and transliterated to a running Rust host that is differentially pinned to the mechanized semantics; on top of it, a hygienic-macro Lisp with a self-interpreting metacircular evaluator closes the loop the paper opens: a reflective tower whose ground is a theorem.

ACM Reference Format:
Stefano Palmieri. 2026. WispyScheme: Metacircularity as a Theorem: Functional Pearl: one law, six machines, and an eight-element algebra. 1, 1 (August 2026), 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 THE LAW EVERYONE ASSUMES

McCarthy defined Lisp in Lisp [McCarthy 1960]: a page of S-expressions, eval and apply, and the discovery that a language can carry its own definition. Sixty-five years later the trick is infrastructure. The Scheme reports specify an eval procedure and a quote form [Shinn et al. 2013], and every conforming implementation satisfies the law that connects them:

$$\mathsf{eval}\ \ulcorner p \urcorner \;=\; \mathsf{run}\ p$$

— evaluating the quotation of p computes what p computes.

Ask where this law comes from, however, and the answer is always the same: it is *stipulated*. The evaluator is written so that the law holds; the law is then cited to justify the evaluator. Metacircular definitions are informative but circular [Reynolds 1972]: the tower of interpreters bottoms out in an implementation whose own quote/eval discipline is asserted, not derived. Verified Lisps tighten the screws without changing the shape — Milawa, Jitawa, and CakeML [Myreen and Davis 2011; Davis and Myreen 2015; Kumar et al. 2014] prove implementations correct *against* a semantics in which the reflective laws are definitional. Somebody, somewhere, always writes the law down first.

This pearl derives it. We exhibit a small Scheme dialect, WispyScheme, whose $\mathsf{eval} \circ \mathsf{quote}$ law is a theorem with non-trivial content: its proof is a structural induction whose base case is a single kernel-checked fact about an independently certified 8×8 operation table. Perturb one cell of the table and the proof breaks at the base case; keep the table and the law propagates to programs of every size. The evaluator does not stipulate the law; the algebra imposes it.

Author's address: Stefano Palmieri, Independent researcher,, .

2026. XXXX-XXXX/2026/8-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

One derivation would be a curiosity. The pearl's actual subject is what happens next: we rebuild the evaluator five times — adding environments (§4), closures and fuel (§5), first-class continuations (§6), a mutable store (§7), and pairs with dispatch (§8) — ending in a call-by-value sequent-calculus (CESK) machine. The law survives every restructuring *verbatim*, with the same base case, and each extension is proved conservative over the one before. We keep swapping the machine out from under the law; the law keeps holding onto the same eight-element fact.

Everything is mechanized in Lean 4: 98 theorems across the six machine files, zero sorry, and — checked with `#print axioms` — no axiom beyond propositional extensionality: no classical choice, no quotient soundness. The mechanized machine is transliterated to a running Rust host — the Kamea machine[1] — pinned to the Lean semantics by differential testing (§10), and on top of the certified core a recognizable Lisp grows — hygienic macros, recursion, a REPL — culminating in a metacircular evaluator that interprets its own quoted source: a reflective tower, grounded (§11).

Contributions. A metacircularity theorem whose base case is an independently certified algebraic law (§3); a six-rung ladder of machines, each conservative over the last, carrying that theorem to a full CESK machine (§§4–8); a certified homoiconicity theorem — every program's quotation is constructible by a program (§8); and a running artifact in which the ladder's top is a self-interpreting Lisp whose adequacy is tested against the certified semantics (§§10–11).

2 A FOUND OBJECT: THE ALGEBRA WITH QUOTE AND EVAL INSIDE IT

The pearl needs one imported artifact, which we treat as a found object and do not re-derive here: an 8×8 operation table over elements $\{0, \ldots, 7\}$.

```
      0  1  2  3  4  5  6  7
0 [  0, 0, 0, 0, 0, 0, 0, 0 ]   halt-true   (absorber)
1 [  1, 1, 1, 1, 1, 1, 1, 1 ]   halt-false  (absorber)
2 [  0, 0, 5, 6, 7, 2, 3, 4 ]   quote
3 [  0, 1, 5, 6, 7, 2, 3, 4 ]   eval
4 [  0, 0, 5, 7, 6, 2, 4, 3 ]   shift
5 [  0, 0, 1, 1, 1, 0, 0, 0 ]   data?
6 [  0, 0, 0, 0, 0, 1, 1, 1 ]   judge?
7 [  0, 0, 0, 0, 1, 0, 0, 1 ]   shift?
```

Writing $a \cdot b$ for the table entry at row a, column b, the laws that matter for this paper, all kernel-checked in Lean, are:

- *Absorbers*: $0 \cdot x = 0$ and $1 \cdot x = 1$ — two halt channels (accept and reject).
- *Extensionality*: distinct elements have distinct rows. (In §11 this austere axiom becomes an equality predicate.)
- *Retraction*: on the six non-absorber elements, $3 \cdot (2 \cdot x) = x$ and $2 \cdot (3 \cdot x) = x$ — elements 2 and 3 are a quotation and its inverse, internal to the table.
- *Introspection with forced negation*: element 5 answers in the halt channels and answers *oppositely* on x and $2 \cdot x$ — the table can ask "is this a quotation?"
- *Duality pairing*: $2 \cdot 2 = 5$, $2 \cdot 3 = 6$, $2 \cdot 4 = 7$ — each operator's code is its judge.

The table is derived, not designed: the full law set admits exactly 228 tables, of which this is the lexicographically minimal one, extracted by SAT enumeration and then certified in Lean. The companion repository holds that derivation together with the theory it sits in — an independence result for the three reflective capabilities and a family of impossibility theorems [Palmieri 2026a]. Here we need three of those impossibility results, stated informally:

[1] A *kamea* is a talismanic number square: a small table carried as an object of power. The machine is named for its table.

The completeness wall. In *every* applicative structure with two absorbers and the classifier discipline above — finite or infinite — some right translation is not a left translation: there is a b such that no a satisfies $a \cdot x = x \cdot b$ for all x. Hence no such structure is combinatorially complete, and in particular none carries a jointly adequate (S, K) pair (d_leaves_a_column_unnamed in the companion development; the obstruction is counting-free and survives at every cardinality). A structure that can judge itself cannot name all of its own transformations — and yet the quote/eval retraction above fits comfortably inside this wall. This is the single theorem the rest of the paper is shaped around.

K-infinity. The finite sharpening: no finite table contains a K combinator at all, so iteration and universality cannot live inside the algebra. Whatever runs programs is an *external driver loop* whose only semantic step is a table lookup.

The collapse wall. A user-facing quote in the Lisp sense maps everything — code and data alike — into data, and a table whose quotation collapses in this way provably cannot also contain an internal eval. Hence user-level quote and eval must be *driver-level* operations, underwritten by (but distinct from) the table's internal duality.

Together the walls force a two-level architecture — table as microcode, driver as machine — and the rest of this pearl lives at the driver level. Two further walls (no faithful internal pairing; no internal recognizer of a faithful constructor) will place pairs and type predicates when we reach §8. In slogan form: Lisp is what universality looks like from inside a system that can judge itself. Quotation survives the walls; S and K do not; and the ladder that follows is an account of everything that must, therefore, live outside.

3 RUNG 0: METACIRCULARITY IN MINIATURE

The minimal driver has atoms and application, and its evaluator has exactly one semantic rule — the table:

```
inductive Prog | atom : Fin 8 -> Prog | app : Prog -> Prog -> Prog
inductive Val  | elem : Fin 8 -> Val  | cell : Val -> Val -> Val

def run : Prog -> Val
  | .atom a  => .elem a
  | .app f x => match run f, run x with
    | .elem a, .elem b => .elem (dot a b)
    | _, _             => .elem 0     -- errors cut to a halt channel
```

Quotation routes atoms through the table's internal duality — rows 2 and 3 — with the two absorbers representing themselves (the reader may recognize Scheme's self-evaluating literals):

```
def qatom (a : Fin 8) : Fin 8 := if a = 0 \/ a = 1 then a else dot 2 a
def eatom (a : Fin 8) : Fin 8 := if a = 0 \/ a = 1 then a else dot 3 a

theorem eatom_qatom : forall a, eatom (qatom a) = a := by decide
```

That one-line theorem — discharged by decide, i.e. by the kernel running the table — is the pearl's load-bearing wall. It is nothing other than the table's certified retraction law plus absorber self-representation. Compound programs quote structurally into inert heap cells (quoteD), decoding inverts (decodeD), and the metacircular law falls out of a structural induction whose leaves are eatom_qatom:

```
theorem decode_quote (p : Prog) : decodeD (quoteD p) = some p
theorem eval_quote   (p : Prog) : evalD (quoteD p) = run p
```

This is the Scheme reports' defining law for eval, derived rather than assumed. The dependency is real and brittle in exactly the right way: edit any cell of rows 2 or 3 and eatom_qatom fails; the law's truth flows from the algebra, through the induction, to programs of every size.

It is worth being exact about where the content of this theorem lives, because the induction itself is deliberately unremarkable. The compound layer is definitional: cells quote structurally and decode by structural recursion, and the pairing wall left no other design available. Every gram of semantic content therefore sits at the atoms — and there the clauses do not stipulate a round trip; they route through rows 2 and 3 of a table that was certified before this file existed, found by an enumeration that could have come back empty. The claim is not that the proof is clever; it is that its base case is imported rather than written — and §12 prices exactly what the import cost.

Rung 0 is, of course, a toy. The pearl's claim is that it is a toy with the right skeleton: *adequacy at the leaves via a certified table law, congruence everywhere else.* Each of the next five rungs adds a feature that real evaluators have and toy proofs usually lack — and the skeleton carries.

4 RUNG 1: ENVIRONMENTS, AND ADEQUACY IS STATIC

The Scheme reports' eval takes two arguments: an expression and an environment. So does ours, from this rung on. Programs gain de Bruijn variables, run and evalD gain an environment ρ, and the law becomes

$$\forall \rho\, p.\ \ \mathsf{evalD}\ \rho\ (\mathsf{quoteD}\ p)\ =\ \mathsf{run}\ \rho\ p.$$

The proof's shape explains why the environment costs nothing: *representation adequacy is static.* The lemma decode_quote mentions no ρ at all — quotation and decoding never consult the environment; only the semantic step does. The law is uniform in ρ because the representation layer cannot even see ρ.

Two design facts arrive with this rung, both dictated rather than chosen. First, with two compound syntax classes (application, variable), representations must carry *tags*; per the recognizer wall, tags live in heap cells, never in instruction identity — this is why pair? in a real Lisp inspects the heap. Second, quoted de Bruijn indices are unary numerals built from shift cells, so one more shift cell on a representation *is* one more binding skipped in the environment (quote_var_succ, shift_cell_skips_binding) — the table's certified renaming operator marking exactly the things renaming acts on.

Finally, conservativity: variable-free programs embed, and under every environment the extended driver computes exactly the minimal driver's value (run_embed). This pattern — extend, then prove the extension changes nothing it does not mention — repeats at every remaining rung, and we will stop remarking on it except where its form improves.

5 RUNG 2: CLOSURES, AND THE FUEL THAT β FORCES

Programs gain λ; values gain closures; application of a closure is β — and evaluation is no longer total. K-infinity said universality lives in an external loop; this is the rung where the loop's nontermination becomes real. run becomes fuel-indexed: none means the loop was cut short; in-band errors still cut to a halt channel. The law strengthens to:

$$\forall\, \mathit{fuel}\ \rho\ p.\ \ \mathsf{evalD}\ \mathit{fuel}\ \rho\ (\mathsf{quoteD}\ p)\ =\ \mathsf{run}\ \mathit{fuel}\ \rho\ p$$

uniformly in fuel — so a program and its quotation *converge together, diverge together, and agree on every value.* This is the precision partiality demands of the metacircular law, and it comes for free from the same static-adequacy structure.

Three theorems put teeth in the fuel. run_mono: a produced value is stable under more fuel, so none only ever means "not yet." Omega_diverges: the term $\Omega = (\lambda x.\, x\, x)(\lambda x.\, x\, x)$ runs to none at *every* fuel — divergence is a theorem, not an anecdote, and the driver loop provably cannot

be internalized. And a small delight, eval_quote_duality_demo: applying $\lambda x.\, x \cdot x$ to the quote instruction and running it through evalD ∘ quoteD computes data? — the table's duality pairing ($2 \cdot 2 = 5$), reached by β-reduction through the quotation machinery, closed by rfl.

6 RUNG 3: THE MACHINE BECOMES SYSTEM L

To host first-class continuations we replace the big-step evaluator with a machine in the CEK family [Felleisen and Friedman 1986], and here an old analogy becomes definitions. Curien and Herbelin's $\bar{\lambda}\mu\tilde{\mu}$-calculus presents computation as *commands* — a producer cut against a consumer [Curien and Herbelin 2000]; its call-by-value discipline, run as an abstract machine, is exactly a CEK machine. Our states are commands $\langle \mathit{focus} \parallel \mathit{kont} \rangle$, and four one-line theorems, each closing by rfl because the correspondence is now definitional, say the rest:

- step_halt: halting is a cut against the toplevel co-constant — the machine's accept channel is the algebra's absorber, doing its original job.
- step_mu: callcc is binder-form μ: it binds the current consumer at de Bruijn index 0 and runs its body against that same consumer.
- step_throw: invoking a captured continuation cuts the value against the captured consumer, *discarding* the current one.
- step_beta: β re-enters the body under the *same* continuation — proper tail calls are machine shape, not an optimization. (A Scheme requirement obtained structurally.)

The metacircular law survives the restructure verbatim (fuel now counts machine steps), because — again — adequacy is static: decodeD rejects closures and reified continuations, so representations never mention consumers. Procedures that cannot be write-n and read back are usually an implementation grumble; here they are a structural property of the driver.

Conservativity earns its keep at this rung: it is a genuine big-step to small-step correspondence. The simulation theorem (machine_sim) is proved continuation-polymorphically — if returning the embedded value to k eventually answers w, then evaluating the program in k eventually answers w — and machine determinism closes the other side on μ-free programs. Ω's divergence also sharpens into something finite: the machine enters a certified *five-state cycle* (machine_delta_cycle), so nontermination is exhibited as a checkable loop rather than an infinite regress.

7 RUNG 4: THE STORE, THREADED BUT NEVER CAPTURED

The S of CESK: locations, allocation, read, write. The structural fact this rung certifies is an asymmetry that every Scheme implementor knows and no Scheme document proves: *environments are captured, the store is threaded.* Closures carry their environment and never a store; every machine step carries the one store forward. Two consequences become theorems:

- eval_quote_mutation: a write performed in argument position is observed by the function body — no environment connects them; the store arrived by threading.
- step_throw (store form): the same σ appears on both sides of the throw rule, so jumping backward through a captured continuation does not undo mutations. Scheme's call/cc-plus-set! semantics, as one rfl.

Conservativity strengthens once more, to a *lockstep bisimulation*: on store-free programs the machine, carrying any store, performs exactly the previous machine's step and never touches σ; at the loop level the two machines' answers are equal under `Option.map` — values and divergence in a single equation, no determinism argument needed, with Ω's divergence transferring for free. (A practical dividend, outside the theorems: because the Kont type provably has no store component, the Rust host of §10 may hold its store as a single mutable vector. The representation choice is licensed by the shape of a type.)

8 RUNG 5: PAIRS, DISPATCH, AND CERTIFIED HOMOICONICITY

The final rung adds structured data — cons, car, cdr, the recognizer pair?, and the conditional if — and every piece lands where the walls said it must. cons builds heap cells (the pairing wall forbids faithful pairing inside the table); pair? reads heap structure, never instruction identity (the recognizer wall); if is a machine form (branching is not a table capability), with the reject absorber as the only false value. With this rung the tag space closes with pleasing exactness: the six non-absorber elements now tag the six compound syntax groups, none unused, none missing.

The headline is what cons does to quotation. Quotations are cells; cons builds cells; therefore *programs can construct quotations*. Three theorems chain:

```
def build : Val -> Option Prog        -- constructor program of a datum
theorem build_sim   : build v = some p -> (running p computes v)
theorem build_quote : forall q, exists p, build (quoteD q) = some p

theorem programs_build_their_own_quotations :
  forall q env sto, exists p fuel,
    runM fuel env sto p = some (quoteD q)
```

For *every* program q there is a program that computes $\ulcorner q \urcorner$ — in any environment, with any store, which it leaves untouched — and composing with the metacircular law, eval of the built quotation runs q. Code as data and data as code, both directions theorems. A concrete instance closes by rfl: running cons 3 (cons 5 6) produces exactly $\ulcorner 2 \cdot 3 \urcorner$, and evaluating that cell is running $2 \cdot 3$.

9 ONE LAW, SIX MACHINES, ONE BASE CASE

Table 1. The ladder. Every row's law is proved by the same induction shape; every base case is eatom_qatom; every rung is proved conservative over the one above it.

rung	machine	the law
0	big-step, atoms + application	evalD (quoteD p) = run p
1	+ environments	$\forall \rho$
2	+ closures, fuel	$\forall$ *fuel* ρ *(uniformly: diverge together)*
3	CEK / System L commands, μ	$\forall$ *fuel* ρ *(fuel = machine steps)*
4	CESK: + store	$\forall$ *fuel* $\rho\ \sigma$
5	+ pairs, dispatch	$\forall$ *fuel* $\rho\ \sigma$, quotations constructible

Table 1 is the pearl. Six machines of increasing realism; one law, strengthening but never changing shape; one base case, a single kernel-checked fact about an eight-element table. The usual metacircular story is a tower of trust that bottoms out in an implementation's say-so. This one bottoms out in eatom_qatom, and eatom_qatom bottoms out in sixty-four table cells the kernel can check by running them.

The mechanization is 98 theorems across six Lean files, none depending on any axiom beyond propositional extensionality. The repository they sit in — the algebra, the walls, the enumeration — is about 435 theorems, zero sorry, with a stratified trust base worth stating exactly: the completeness wall depends on no axioms at all; the ladder and the finite witnesses need only propext; the frame-derivation and structure theorems additionally use classical choice through Mathlib's finiteness machinery; and the $N = 10$ enumeration witnesses are checked by native_decide, which trusts Lean's compiler. The law this pearl is about never leaves the first stratum.

10 ABOVE THE CORE, A LISP

Theorems about machines are best believed when the machine runs. The Lean development is transliterated, rule for rule with citations in comments, to a Rust host, the Kamea machine [Palmieri 2026b]; a reference oracle executes the *certified* machine via Lean's interpreter (its store-observing loop proven to agree with the certified loop); and a differential harness generates seeded programs and demands bit-identical results — value, in-band error, or divergence at exact fuel, plus the final store — between the two. The standing runs hold at 24,000 cases with zero mismatches. The Rust core is not verified; it is *pinned*, and every claim below should be read through that chain: surface behavior → Rust machine → differential pinning → certified semantics → the table.

On the pinned core a recognizable Scheme grows — WispyScheme, the language of this paper's title — and the striking thing is how little of it required new semantics:

- **Recursion** is Landin's knot on the store rung: letrec desugars to fresh locations, η-expanded forwarders, and backpatching. Mutual recursion runs through one shared knot. Zero new core forms.
- **Lists and numerals** are the certified encodings surfaced: nil is the accept absorber — the same terminator the certified de Bruijn numerals already use — and succ is one more shift cell. zero? is pair?; pred is cdr.
- **Hygienic-enough macros**: a syntax-rules expander runs driver-side, before compilation, so macros add no core semantics. Template-introduced binders are freshened with names the tokenizer cannot produce; the classic or/swap! capture examples are regression tests. (Free template identifiers resolve at use sites — the known gap to full hygiene, stated rather than fudged.)
- **Element equality needs no primitive.** Extensionality — distinct elements have distinct rows — means decision trees of *applications* separate all eight elements, and the whole language of this section, metacircular evaluator included, runs on that discipline alone. The axiom that powered the algebra's independence proofs resurfaces as the operational fact that the language can identify its own atoms by how they behave. (An eqv? core form has since been added as a seventh, post-paper rung — same law, same base case, same conservativity — and its correctness on elements is certified to *agree* with extensionality: two instructions are eqv?-identical iff their rows coincide. The primitive adds speed; the decision trees remain the proof that it adds nothing else.)

At the prompt, `(fib 6)` evaluates to `#8` by a doubly recursive letrec over unary numerals, every step of it a machine transition bottoming out in table lookups.

11 THE TOWER, GROUNDED

The oldest demo in Lisp is the metacircular evaluator; the oldest anxiety about it is what it rests on [Smith 1984; Wand and Friedman 1988]. We close the pearl by closing the loop: a metacircular evaluator for the full core language, written in the surface language, adequacy-tested against the certified machine, and stacked two levels high.

The evaluator meta is a *closed* surface program — no free globals, hence itself quotable, which is what makes the tower possible. It interprets quotations of all thirteen core syntax classes with a tagged value representation whose tags are elements (quo for elements, evl for closures, shf for continuations, data? for locations, judge? for cells); its tag and syntax dispatch is built entirely from the extensionality decision trees of §10; object callcc absorbs into host callcc in the classic direct style; the object store shares the host tape exactly as the Scheme reports' eval shares the store; and environment lookup walks the quotations' own shift-cell numerals, which are the prelude's numerals.

Adequacy: for a 17-program corpus covering every syntax class — shadowing, escapes, mutation sequencing, and recursion through letrec's knot, meaning the evaluator interprets the refs, forwarders, and backpatching as object code — reifying meta $\ulcorner p \urcorner$ equals running p, as exact value equality on the pinned machine. This is run $(E \ulcorner p \urcorner) =$ run p differentially; its theorem-shaped form (interpreter adequacy in Lean) is the natural sequel to this pearl, not part of it.

The tower: evaluate

```
(reify (reify (meta '(META (cons 3 (cons 5 6))))))
```

where META is the evaluator's own source. Level 1 interprets the evaluator; level 2 interprets a program. The elegant part is the level boundary: the inner quotation is cons-built *inside* the interpreted world, and by construction, evaluated data *is* the injected representation — the certified homoiconicity theorem of §8 doing load-bearing work at the join between levels. Two reifications unwrap the two representation layers; the answer is judge?, which is $2 \cdot 3$, which is what the two-node object program denotes. The whole tower runs in milliseconds — small tables make cheap universes — and every step of both levels is a transition of the machine whose law is the theorem of §3.

McCarthy's discovery was that the language can define itself. The tower's traditional embarrassment is that the self-definition rests on turtles. Here the turtles bottom out: two levels of interpretation, one pinned machine, one certified law, one table.

12 WHAT IS FORCED, AND WHAT IS CHOSEN

A pearl about derivation owes the reader a precise account of what was derived. The claims, sorted:

Theorems. The atomic quote/eval duality and its retraction; the two-level architecture (walls); the necessity of heap tags; the metacircular law at every rung; conservativity of every extension; fuel monotonicity and Ω's divergence; the store/environment asymmetry; constructibility of all quotations; the System L correspondences of §6, each by rfl. And part of the algebra's own frame: an introspector that can *observe* quotation — answer differently on x and $2 \cdot x$ — forces the class-swapping world, and together with a faithful third operator forces $n \geq 8$, sharply (stack_a_frame_min); and the eval side of the law set is derivable from the quote side — commutation and judge-closure both transport through the retraction (eval_comm_of_quote_comm, eval_closure_of_quote_closure, the latter by a finite-orbit argument). Further: every faithful operator has finite core order (reversibility of renaming is free), that order is necessarily even in the swap world (so the involution law is the choice of the *minimal* order — a tie-break, not an axiom), and the introspector's realized quote-orbit is automatically closed under both quote and eval (QuoteOrbit.lean). What survives as genuine choice: that hygiene means commuting with quote (a definition), closure for judges outside the orbit, shift's distinctness, and two tie-breaks. The choices of world, of size, of the eval side, of reversibility, and of the order's parity were choices only until they were derived.

Engineering, and labeled as such. Which element tags which syntax class; the compound encodings and numerals; frame layouts; which value setref returns; the surface language's desugarings. The compound layer above the atoms is standard programming-language metatheory, cleanly done — its connection to the algebra runs through the atoms and the walls, not through every constructor.

The novelty claim, exactly. We know of no other system in which the eval/quote law of a running language is a *consequence* of an independently certified structure, rather than a property stipulated by — or verified against — the language's own semantics. We would welcome counterexamples.

13 RELATED WORK

McCarthy's metacircular definition [McCarthy 1960] and Reynolds's analysis of definitional interpreters [Reynolds 1972] frame the problem this pearl inverts. Our ladder is a certified cousin of Danvy et al.'s functional correspondence [Ager et al. 2003] — where they derive machines from evaluators by program transformation, we derive the *law* once and prove each machine conservative over the last; the CEK/CESK family is Felleisen and Friedman's [Felleisen and Friedman 1986], and the command reading is Curien–Herbelin's [Curien and Herbelin 2000]. Verified Lisps and MLs — Jitawa [Myreen and Davis 2011], Milawa's trusted core [Davis and Myreen 2015], CakeML [Kumar et al. 2014] — verify implementations against semantics in which reflective laws are definitional; the contrast is the pearl's point rather than a gap in theirs. Reflective towers are Smith's [Smith 1984], demystified by Wand and Friedman [Wand and Friedman 1988]; our contribution to that conversation is a ground floor. The algebra itself, its independence theorems, and its impossibility walls are developed in the companion repository [Palmieri 2026a]; the contrast between the walls and the functional completeness of partial combinatory algebras [van Oosten 2008] is developed there as well. The Rust host's engineering owes debts to Ribbit's tagged-cell discipline [Yvon and Feeley 2021].

14 CODA: THE DEFINITION SCHEME NEVER HAD

There is a wrong sentence nearby that this pearl must not say, and a right one it exists to say. The wrong one: *this is the one true Scheme.* Truth is not a predicate that applies to definitions and tie-breaks, and our artifact, with its eight atoms and unary arithmetic, does not run R7RS. The right sentence has a shape mathematics knows well. Nobody claims the axioms of a complete ordered field *are* the real numbers; everybody accepts that they say what the reals *are* — a short list of demands with a canonical model, unique up to the appropriate equivalence. Before Dedekind, the reals existed as practice: geometers used them fluently and could not say what they were. The characterization did not replace the practice. It told the practice what it had been talking about.

Scheme has existed for fifty years as practice — reports, implementations, folklore about why quote and eval and hygiene are the way they are. What the development in this pearl and its companion amounts to is the Dedekind step: an intensional definition the practice never had. Tally the ledger of §12 and what remains outside the theorems is exactly this: *one wager* (a finite system that judges itself, carrying its own representations under a type discipline); *three definitions* (what representation, quotation-observing introspection, and hygiene mean); *two completeness demands* (the introspector's orbit is askable; the one judge outside it closes); and *five conventions* for picking the canonical representative. Everything else — the two-level architecture, the swap world, the size, the tags' necessity, the eval side, the parity of renaming, the metacircular law itself — is a theorem. Scheme-nature is the class of models of that short list; the table of §2 is its least model; and the price list of every convention used to pick it is finite, named, and public.

Two objections deserve their answers on the record. *You distilled the axioms from Lisp, so of course Lisp came out*: the rebuttal is the compression ratio and the surprises. The axioms are a few lines; the swap world, the heap tags, N = 8, and the free eval side were not put in — and the walls could have falsified the enterprise. Nobody designing axioms to flatter Lisp would design the theorem that self-judgment excludes S and K. When axioms fight back, they are describing something. *The definition covers the skeleton, not the flesh*: correct, and "complete ordered field" does not fix decimal notation either. But note which test has been running throughout this pearl: at every rung, some behavior specific to the Scheme reports fell out as a theorem — §6.12's law, tail calls as machine shape, call/cc not restoring the store, unwritable procedures. Each recovery is evidence that the characterization captures the language and not a toy resembling it; the day an essential behavior

cannot be recovered without a new wager, the definition fails, and that falsifiability is what makes it worth stating.

McCarthy showed the language could define itself. The tower of §11 shows the self-definition can be grounded. This coda states what grounds the ground: not the one true Scheme — the definition Scheme never had.

ARTIFACT

The Lean development (the six-rung ladder, the algebra, and the walls; Lean 4.28, zero sorry) and the Kamea machine (the Rust host: machine, differential harness, Lisp, metacircular evaluator; cargo test runs everything in this paper) are available at [Palmieri 2026a,b].

REFERENCES

John McCarthy. Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM* 3(4), 1960.

Alex Shinn, John Cowan, and Arthur A. Gleckler (eds.). *Revised^7 Report on the Algorithmic Language Scheme*. 2013.

John C. Reynolds. Definitional interpreters for higher-order programming languages. *Proceedings of the ACM Annual Conference*, 1972.

Mads Sig Ager, Dariusz Biernacki, Olivier Danvy, and Jan Midtgaard. A functional correspondence between evaluators and abstract machines. *PPDP*, 2003.

Matthias Felleisen and Daniel P. Friedman. Control operators, the SECD-machine, and the λ-calculus. *Formal Description of Programming Concepts III*, 1986.

Pierre-Louis Curien and Hugo Herbelin. The duality of computation. *ICFP*, 2000.

Magnus O. Myreen and Jared Davis. A verified runtime for a verified theorem prover. *ITP*, 2011.

Jared Davis and Magnus O. Myreen. The reflective Milawa theorem prover is sound. *Journal of Automated Reasoning* 55(2), 2015.

Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. CakeML: a verified implementation of ML. *POPL*, 2014.

Brian Cantwell Smith. Reflection and semantics in Lisp. *POPL*, 1984.

Mitchell Wand and Daniel P. Friedman. The mystery of the tower revealed: a non-reflective description of the reflective tower. *Lisp and Symbolic Computation* 1(1), 1988.

Jaap van Oosten. *Realizability: An Introduction to its Categorical Side*. Elsevier, 2008.

Samuel Yvon and Marc Feeley. A small Scheme VM, compiler, and REPL in 4K. *VMIL*, 2021.

Stefano Palmieri. finite-magma-independence: Lean formalization. https://github.com/stefanopalmieri/finite-magma-independence.

Stefano Palmieri. kamea-machine: the certified core's Rust host. https://github.com/stefanopalmieri/kamea-machine.